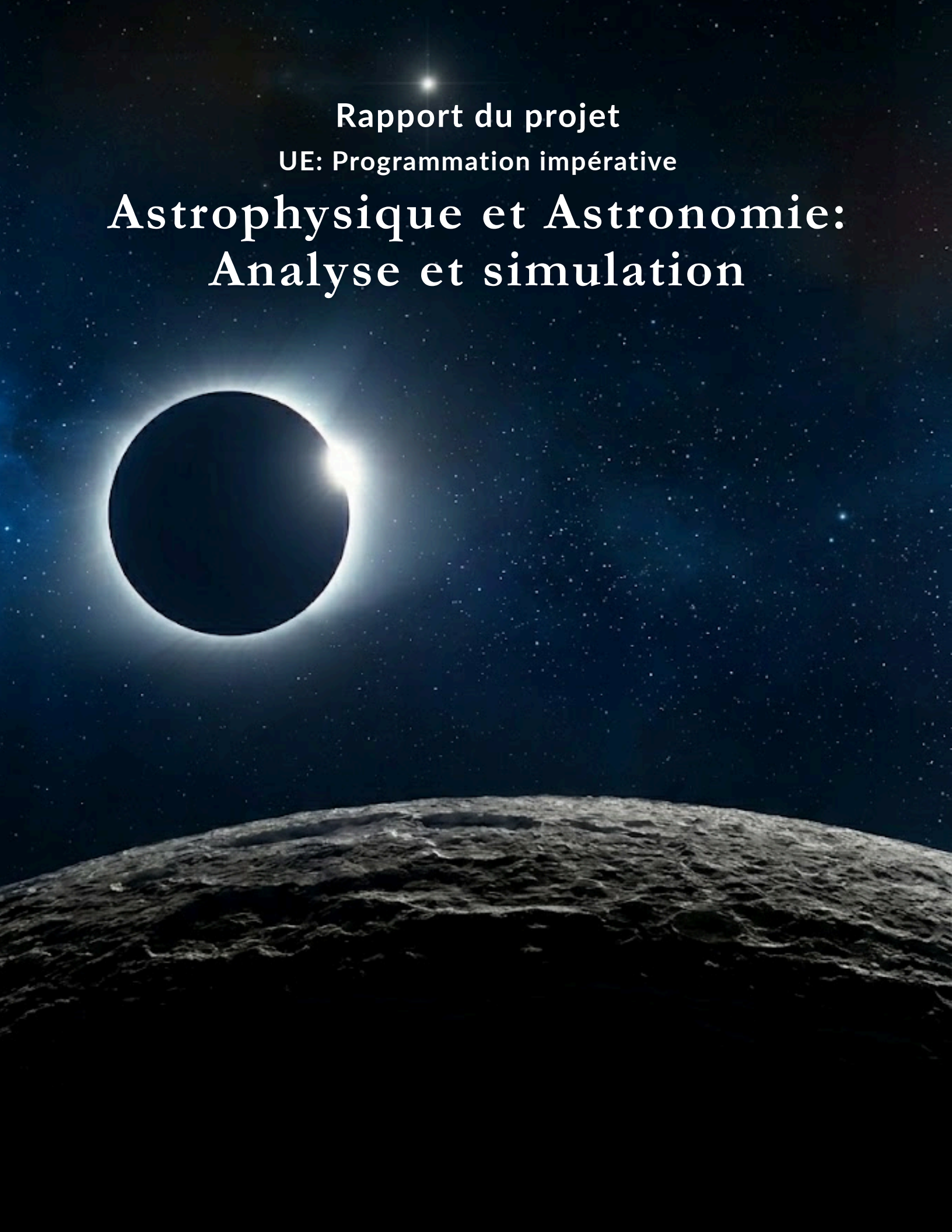


Rapport du projet

UE: Programmation impérative

# Astrophysique et Astronomie: Analyse et simulation



# **Table des matières**

|                                                 |    |
|-------------------------------------------------|----|
| 1.Introduction .....                            | 3  |
| 2.Présentation du projet .....                  | 4  |
| I) Partie Analyse .....                         | 4  |
| II) Partie Simulation .....                     | 13 |
| 3.Choix techniques .....                        | 16 |
| 4. Architecture du code et lois utilisées ..... | 19 |
| 5. Conclusion .....                             | 30 |
| 6. Annexes .....                                | 31 |

# Introduction

Depuis longtemps, l'humanité cherche à comprendre l'univers et comment il fonctionne. Cela est devenu possible avec l'astronomie et l'astrophysique, qui permettent d'étudier les mouvements des astres, les lois physiques qui contrôlent l'espace, ainsi que les relations entre différentes caractéristiques des étoiles et des planètes.

Ces domaines sont très intéressants et attirants car ils nous permettent de découvrir les mystères de l'espace. Aujourd'hui, ces sciences dépendent énormément de l'informatique pour traiter des données, analyser des phénomènes et faire des simulations utiles.

Dans ce projet, réalisé dans le cadre de l'UE informatique 2, nous avons utilisé le langage Python pour analyser des données astrophysiques réelles et de simuler le mouvement des corps spatiaux sous l'effet de la gravité.

L'objectif est d'appliquer les méthodes de calcul et de modélisation pour mieux comprendre certains lois fondamentales de l'univers.

Ce rapport présente les étapes de notre travail, nos recherches, nos choix techniques et théoriques, ainsi que les résultats obtenues et la description détaillée de notre code.

# Présentation du projet

Ce projet comporte deux grandes parties: l'analyse de données astrophysiques réelles et une simulation numérique du système solaire.

## I) Partie analyse

La première partie est consacrée à la vérification de la validité de certaines lois et relations physiques fondamentales, en analysant des données réelles. Nous avons étudié les relations suivantes :

### 1) La troisième loi de Kepler:

Aussi dite “Loi des périodes”, est l'une des lois que nous avons étudié en école plusieurs fois, qui est découverte par Johannes Kepler en 1618. Elle décrit la proportionnalité entre le carré de la période (T) et le cube de demi-grand axe (a) de la trajectoire elliptique de l'astre.

Nous avons vérifié cette loi à partir des données des lunes de Jupiter et des planètes du système solaire, en appliquant une régression linéaire sur la relation  $T^2 = f(a^3)$ .

- Résultats obtenus:

Pour les données des planètes de Jupiter, nous avons calculer tous d'abord la constante théorique qui vérifie  $T^2 = k.a^3$ , et qui vaut:

$$k = \frac{4\pi^2}{GM_j}$$

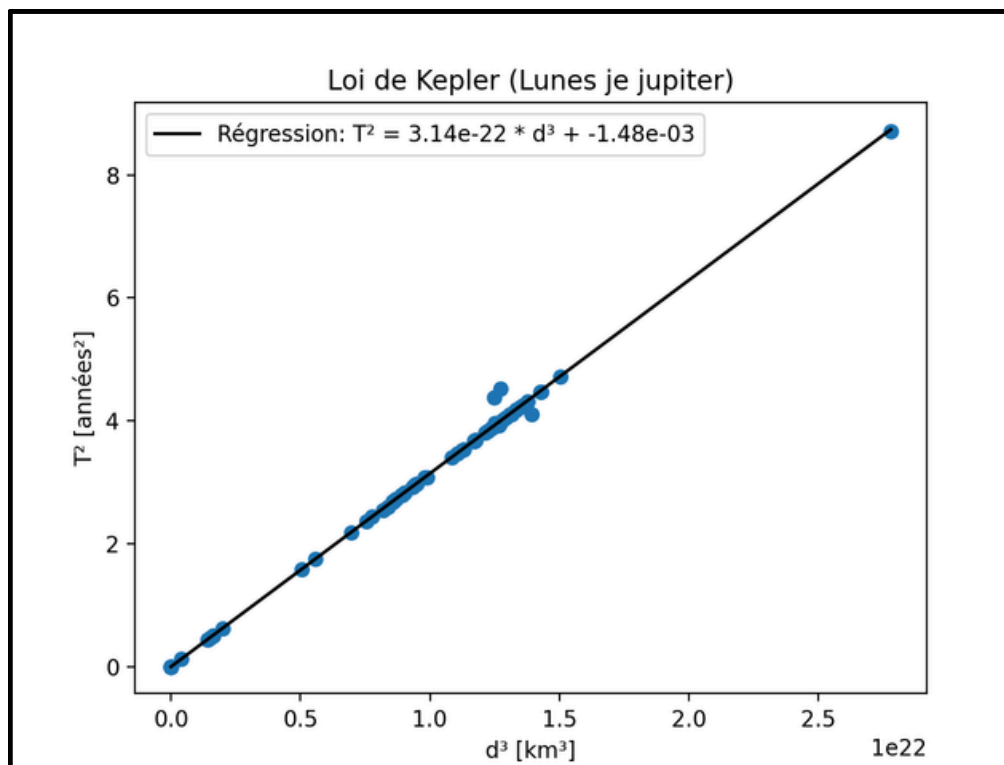
Avec G: constante gravitationnelle ( $G=6.674e-11\text{m}^3\cdot\text{kg}^{-1}\cdot\text{s}^{-2}$ )

$M_j$  : La masse de Jupiter ( $M_j = 1.898e27 \text{ Kg}$ )

Nous avons obtenue par calcul que  $k = 3.1336168e-22$ .

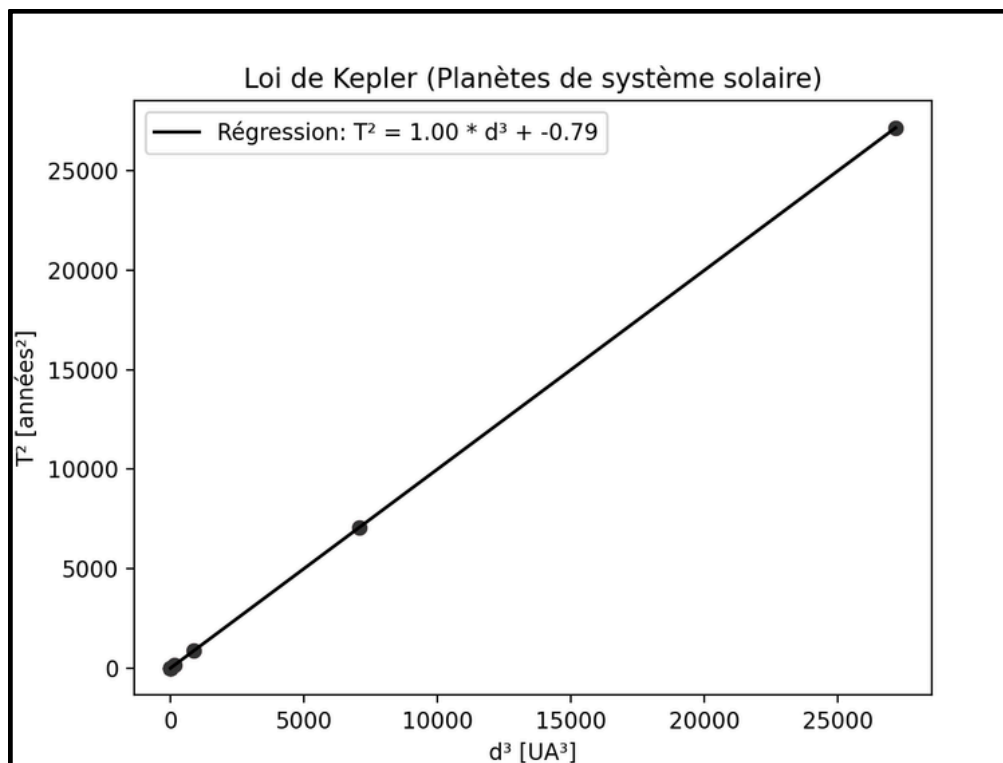
Ensuite, nous avons comparé nos résultats de régression avec celles théoriques. La régression nous a donné une courbe d'équation  $T^2 = 3,14e-22 * a^3 - 0.001$ , ce qui est très proche de la valeur théorique de la constante k.

**Figure 1 : Loi de Kepler — Lunes de Jupiter**



Nous avons appliqué la même démarche aux planètes du système solaire, en utilisant cette fois des unités astronomiques (UA) pour les distances et des années pour les périodes, ce qui simplifie le coefficient théorique  $k$  à 1. La régression linéaire a donné une droite linéaire d'équation  $T^2 = 0.9998 * a^3 - 0.793$  ce qui est en accord avec la théorie et confirme la validité de la 3ème loi de Kepler.

**Figure 2 : Loi de Kepler — Planètes du système solaire**



## 2) Relation entre température et distance:

Cette loi représente le lien entre la température d'équilibre d'une planète et sa distance à son étoile. En effet, plus la planète est proche de l'étoile, elle reçoit plus de l'énergie rayonnante, et donc sa température devient plus élevée.

D'un point de vue théorique, la formule de la température d'une planète en supposant que sa seule source de lumière est son étoile, est :

$$T = \left( \frac{L (1 - a)}{16\sigma\pi d^2} \right)^{\frac{1}{4}}$$

Avec L : Luminosité de l'étoile

$\sigma$  : Constante de Stefan-Boltzmann =  $5.67 \times 10^{-8} \text{ W}\cdot\text{m}^{-2}\cdot\text{K}^{-4}$ , relie la température d'un corps noir (sa source unique de lumière est son étoile) à l'énergie qu'il rayonne.

a -> Albédo de la planète : fraction de la lumière réfléchiée par la planète

d : Distance entre la planète et son étoile

Cette relation indique que la température d'équilibre d'une planète est proportionnelle à  $\frac{1}{\sqrt{d}} \rightarrow d^{-0.5}$  en négligeant les effets de luminosité (L) et l'albédo (a).

- Démarche

Pour vérifier cette relation, nous avons utilisé les données de l'archive officielle de la NASA contenant les propriétés des exoplanètes confirmées. Afin d'éviter les planètes avec des comportements inhabituels, nous avons appliqué deux filtres sur les données :

- Ne garder que les exoplanètes situées à moins de 4 UA de leur étoile, pour enlever les planètes gazeuses très éloignées.
- Ne garder que les exoplanètes de masse supérieure à 5 masses terrestres, pour se concentrer sur des objets suffisamment massifs et éviter ceux qui sont rocheux dont la température dépend fortement de facteurs locaux (atmosphère, albédo).

Pour pouvoir effectuer une régression linéaire sur une telle relation de puissance, nous avons transformé les données en échelle logarithmique :

$$T = C \cdot d^{-0.5} \Rightarrow \log(T) = \log(C \cdot d^{-0.5})$$

$$\Rightarrow \log(T) = \log(C) - 0.5 \times \log(d)$$

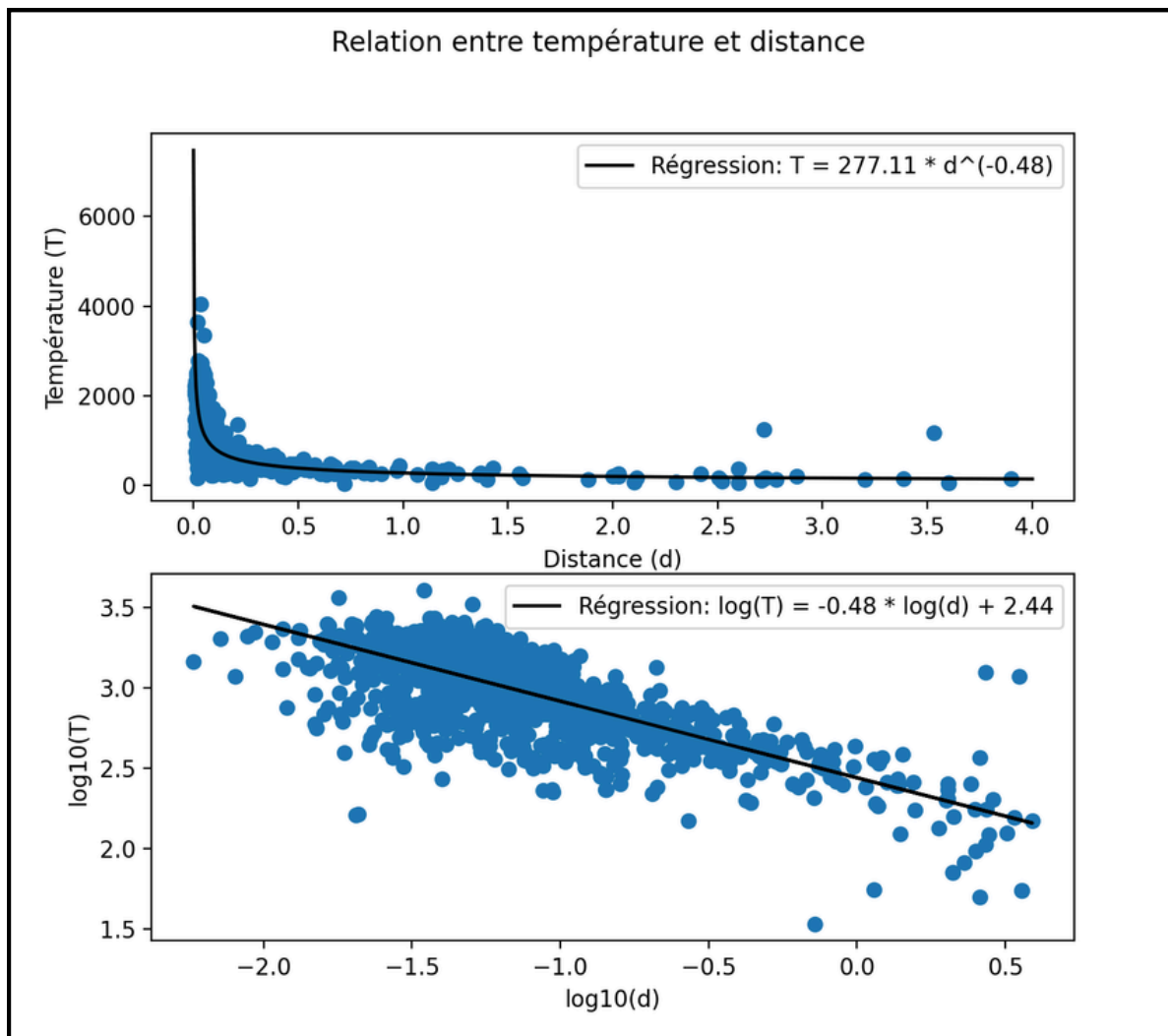
Cette transformation nous permet d'obtenir une relation linéaire à laquelle on peut appliquer la régression de type  $y = ax+b$ .

- Résultats obtenus:

La régression donne la relation  $T = 277.11 \times d^{-0.48}$ , ce qui est proche de la relation théorique  $T \propto d^{-0.5}$ , avec un exposant de  $-0.48$  très proche de  $-0.5$ , ce qui confirme la validité de la relation théorique entre la température et la distance.



**Figure 3 : Résultats d'analyse des données de température et distance des exoplanètes**



### 3) Relation entre le rayon et la masse des étoiles:

La relation masse-rayon est le lien qui associe de manière caractéristique la masse et le rayon d'un objet du fait de sa composition et sa structure.

De façon générale, les étoiles de la séquence principale (qui sont stables) suivent une relation du type suivant:

$$R = M^{\xi}$$

où M et R sont la masse et le rayon de l'étoile considérée en masse solaire et rayon solaire respectivement.

Cependant, on peut distinguer deux régimes (deux valeurs de  $\xi$ ) selon la gamme de masses à laquelle appartient l'étoile, la « rupture » se produit autour de la masse du Soleil.

- pour les étoiles moins massives que le Soleil ( $M < 1$ ):

$$R = M^{0.8}$$

- pour les étoiles plus massives que le Soleil ( $M > 1$ ):

$$R = M^{0.57}$$

- Démarche

En premier temps, nous avons appliqué une transformation logarithmique aux données, vue que c'est une relation de puissance. Cependant, nous avons remarqué que certaines étoiles sont très loin du

nuage des points principal dans le graphique représentant le rayon en fonction de la masse. Ce sont probablement des étoiles peut stables et des cas particuliers. Pour enlever ces étoiles, nous avons utilisé une fonction qui détecte les valeurs aberrantes à l'échelle logarithmique.

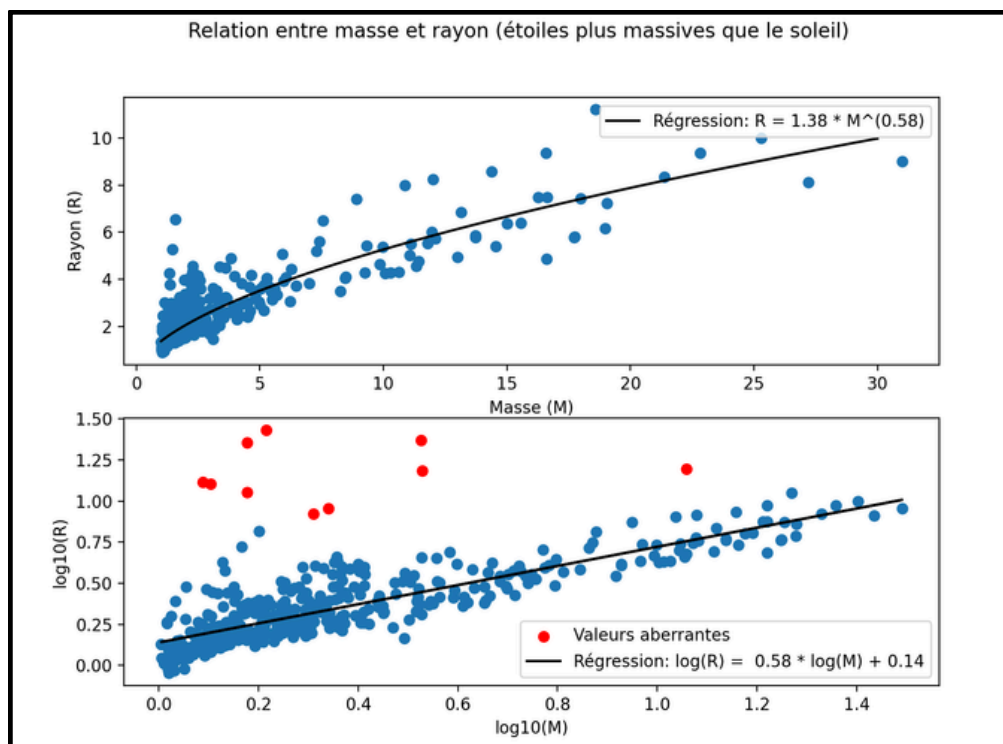
- Résultats obtenus

- 1. Les étoiles plus massives que le Soleil

Résultat de la régression:  $R = 1.38M^{0.58}$

Ce résultat est très proche de la relation théorique:  $R = M^{0.57}$

**Figure 4: Résultats d'analyse de relation en masse et rayon des étoiles plus massives que le Soleil**

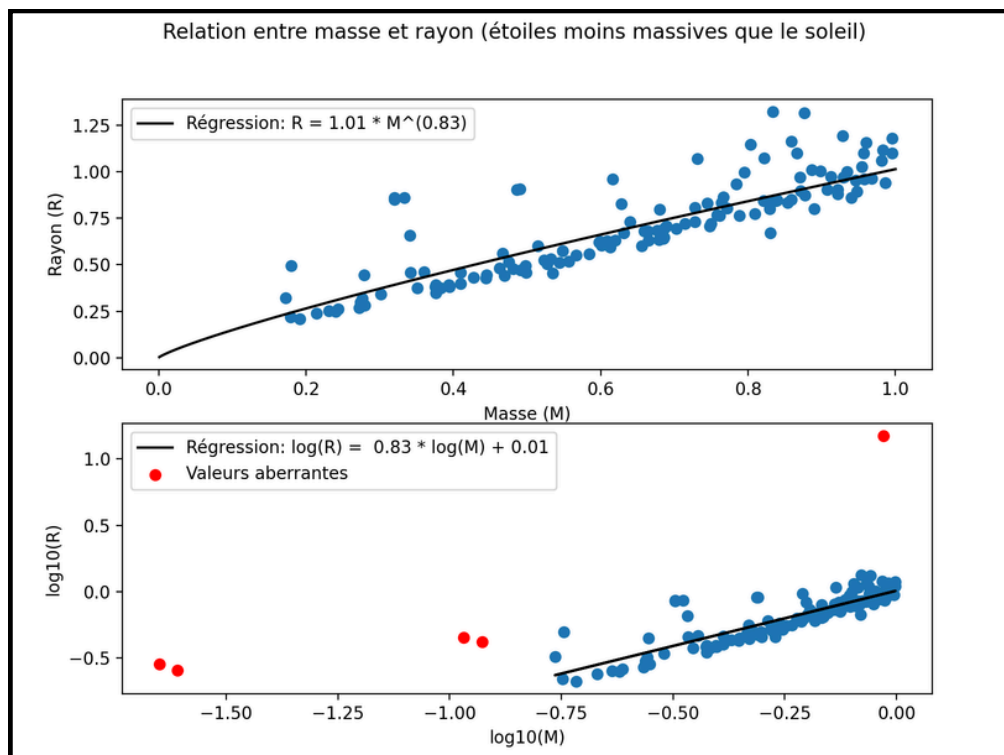


## 2. Les étoiles moins massives que le Soleil

Résultat de la régression:  $R = 1.01M^{0.83}$

Ce résultat est très proche de la relation théorique:  $R = M^{0.8}$

**Figure 5: Résultats d'analyse de relation en masse et rayon des étoiles moins massives que le Soleil**



L'analyse de données réelle montre une tendance très proche de la loi théorique, que ce soit pour les étoiles plus massives que le soleil ou celles moins massives.

## **II) Partie Simulation**

### **1) Présentation générale**

Pour la deuxième partie de ce projet, nous avons réalisé une simulation dynamique des trajectoires des planètes du système solaire sous forme d'une animation. L'objectif de cette simulation est de mieux comprendre de manière visuelle l'évolution des positions des planètes au cours du temps, et l'effet de l'attraction du Soleil sur ces planètes, ainsi que les attractions des autres astres. Elle nous permet également de vérifier la validité de certaines lois comme celle de Kepler, d'observer la nature des trajectoires des planètes, et les différences entre leurs vitesses et périodes.

Notre simulation ne se limite pas aux 8 planètes du système solaire, mais nous avons également ajouté d'autres planètes qui sont générées aléatoirement. Nous avons aussi ajouté une probabilité que des astéroïdes apparaissent. Cela nous permet d'observer la stabilité du système solaire, et comment il réagit face aux perturbations extérieures.

### **2) Fonctionnement de la simulation**

La simulation consiste à calculer la position de chaque objet à chaque pas du temps. Afin de rendre ces calculs

les plus précises possible, nous avons utilisé des données réelles du site de la NASA pour les conditions initiales, et la méthode de Verlet pour calculer les positions des corps au cours du temps. Les détails de cette méthode seront présentés dans la partie <Architecture du code et lois utilisées>.

### **3) Collisions et fusions**

La présence de nouveaux corps extérieurs peut perturber les trajectoires et les vitesses des objets sous l'effet d'attraction gravitationnelle, ce qui augmente les risques des collisions. Ces collisions sont gérées dans la simulation à l'aide du principe de conservation de la quantité de mouvement en fusionnant les objets en contact.

### **4) Gestion de l'affichage dans la simulation**

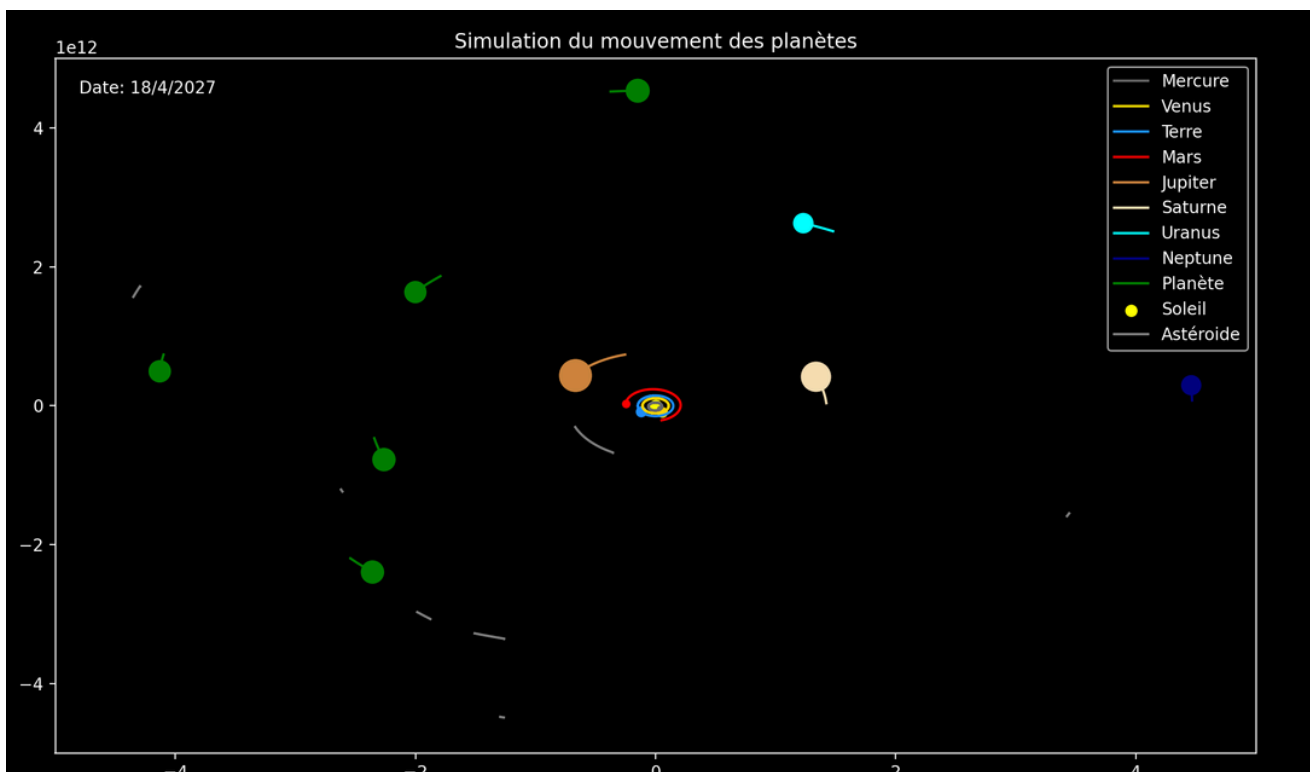
Dans la simulation, le temps est affiché directement en haut à gauche, et est mis à jour en continu. Cela permet de suivre l'évolution du système en connaissant la date actuelle de la simulation.

Les objets sont représentés en tenant compte de leur rayon réel, ce qui permet de respecter les différences de taille entre les corps célestes et de rendre la visualisation plus réaliste.

## 5) Observations

La simulation nous a permis de faire plusieurs observations. Premièrement, nous avons remarqué que les planètes et les astéroïdes plus proches de Soleil tournent plus vite que les autres, leur période est donc faible, ce qui est cohérent avec la 3eme loi de Kepler. Deuxièmement, les trajectoires des planètes ne sont pas parfaitement circulaires, mais décrivent une forme elliptiques, ce qui est conforme à la première loi de Kepler. Concernant les collisions, elles restent rares dans des conditions normales et n'ont pu être observées qu'en augmentant le nombre de planètes aléatoires et la probabilité d'apparition des astéroïdes.

**Figure 6: Capture d'écran illustrant la simulation du système solaire**



# Choix techniques

Dans cette partie nous allons représenter nos choix techniques réalisés lors du développement du projet, ainsi que les raisons ayant motivés ces choix.

## 1) Choix des bibliothèques

- **Matplotlib:** choisi pour sa capacité à représenter des données scientifiques et à générer des simulations dynamiques. Elle nous a permis de créer les graphiques ainsi que l'animation de la simulation du système planétaire.
- **Numpy:** permet d'effectuer simplement, efficacement et rapidement des calculs vectoriels, ainsi que des opérations mathématiques tel que le logarithme, indispensables à la simulation gravitationnelle et aux régressions.
- **CSV:** ce module permet de manipuler des fichiers avec format CSV. Le choix d'utiliser ce format s'explique par la simplicité de sa lecture et par la facilité avec laquelle il permet d'extraire les données nécessaires.
- **Random:** choisi afin d'introduire une composante aléatoire dans la simulation, permettant de générer des astéroïdes et des planètes avec des caractéristiques variées et de rendre le système plus réaliste en ajoutant des perturbations.



## **2) Choix des sources de données**

Les données utilisés dans ce projet sont issues de plusieurs sites scientifiques fiables et spécialisés en astronomie.

- Pour les lunes de Jupiter, les données proviennent d'un dépôt GitHub scientifique contenant des mesures orbitales précises.
- Pour les planètes du système solaire, nous avons utilisé une base de données dédiée aux paramètres orbitaux des planètes.
- Pour les exoplanètes (température, distance et masse), les données proviennent de l'Exoplanet Archive de la NASA, une référence dans l'étude des systèmes planétaires.
- Pour les propriétés des étoiles (masse et rayon), les données sont issues de la base VizieR de l'Observatoire de Strasbourg.
- Les positions et vitesses initiales des planètes dans la simulation sont basées sur les éphémérides du système JPL Horizons de la NASA.

## **3) Transformation en échelle logarithmique**

Pour étudier certaines relations de type loi de puissance, nous avons transformé les données en échelle logarithmique. Cette méthode consiste à prendre le logarithme des grandeurs afin de linéariser la relation. Cela permet de transformer une équation de type  $y = bx^a$  en une relation linéaire de forme

$\log(y) = a \cdot \log(x) + \log(b)$ , ce qui nous permet d'appliquer une régression linéaire.

#### **4) Choix des lois et méthodes physiques**

Dans ce projet, nous avons utilisé plusieurs lois et méthodes physiques afin de modéliser le mouvement des corps d'une manière proche de la réalité. La plus essentielle est la gravitation universelle de Newton, ainsi que la méthode de Verlet qui permet de calculer les positions des objets d'une manière précise. En plus, nous avons utilisé le principe de conservation de quantité de mouvement pour simuler correctement la fusion des corps.

#### **5) Réduction de nombre des positions sauvegardés**

En effectuant des calculs avec de très grands tableaux Numpy, la mémoire de l'ordinateur va être fortement consommée et cela va ralentir l'animation et la performance générale de l'ordinateur. Pour éviter ce problème, nous avons limité les tailles des tableaux Numpy à 500 éléments, ce qui permet de simuler plusieurs corps sur de longues durées tout en conservant de bonnes performances.

# Architecture du code et lois utilisées

## 1) Structure générale

Le code est organisés en deux grandes parties principales:  
**Partie d'analyse de données** (lignes 1--259) **et partie simulation** (lignes 260--600).

## 2) Fonctions clés

| Fonction          | Description                                                   |
|-------------------|---------------------------------------------------------------|
| avg(x)            | Calcule la moyenne d'un ensemble de données                   |
| ecart(x)          | Calcule l'écart-type                                          |
| aberr(x, y, a, b) | Détecte les valeurs aberrantes selon des critères             |
| regression(x, y)  | Effectue une régression linéaire (moindres carrés)            |
| distance(i, j)    | Calcule la distance entre deux corps ( théorème de Pythagore) |

| Fonction                  | Description                                               |
|---------------------------|-----------------------------------------------------------|
| acceleration(j)           | Calcule l'accélération d'un objet j                       |
| generer_aleatoire(choice) | Génère une planète ou astéroïde avec paramètres réalistes |
| fusion(surv, disparaît)   | Fusionne deux objets en collision                         |
| animate(i)                | Fonction principale de la boucle d'animation              |
| leap(i)                   | Détermine si une année est bissextile                     |

### 3) Schémas d'architecture

#### 3.1) Vue d'ensemble des parties

Le diagramme ci dessous illustre l'organisation générale du programme en deux parties:

# Projet

Bibliothèques: numpy • random • matplotlib • csv

## Analyse

### Données:

- Exoplanets.csv
- Solar\_System\_data.csv
- Jupiter\_Moons.csv
- Stars.csv

### Fonctions :

- avg()
- ecart()
- aberr()
- regression()

### Sorties :

- Graphiques de régression
- Coefficients (a, b)
- Validation des lois

## Simulation

### Données initiales:

- Positions
- Vitesses
- Masses
- Rayons

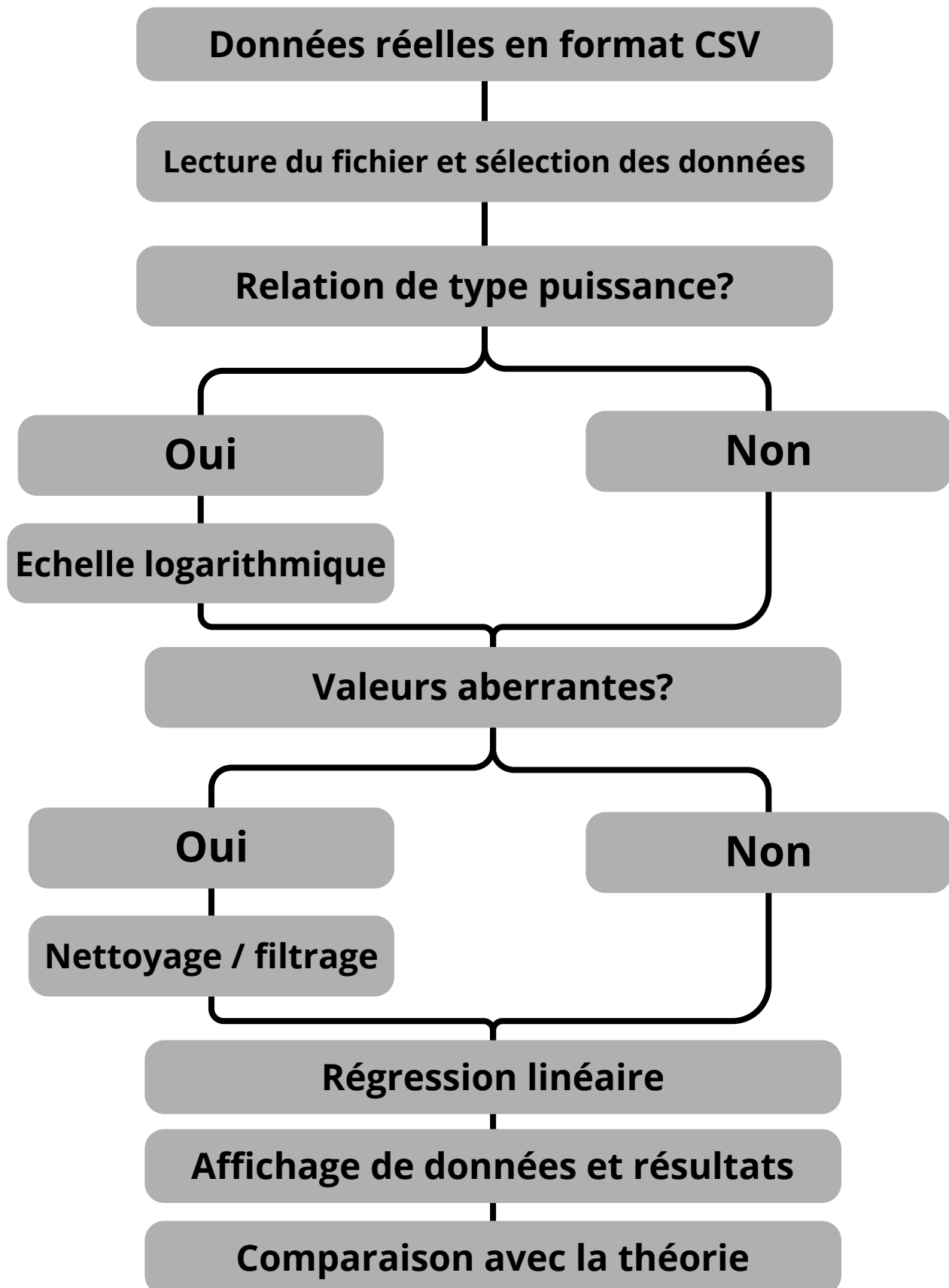
### Fonctions :

- generer\_aleatoire()
- distance()
- acceleration()
- leap()
- fusion()
- animate()

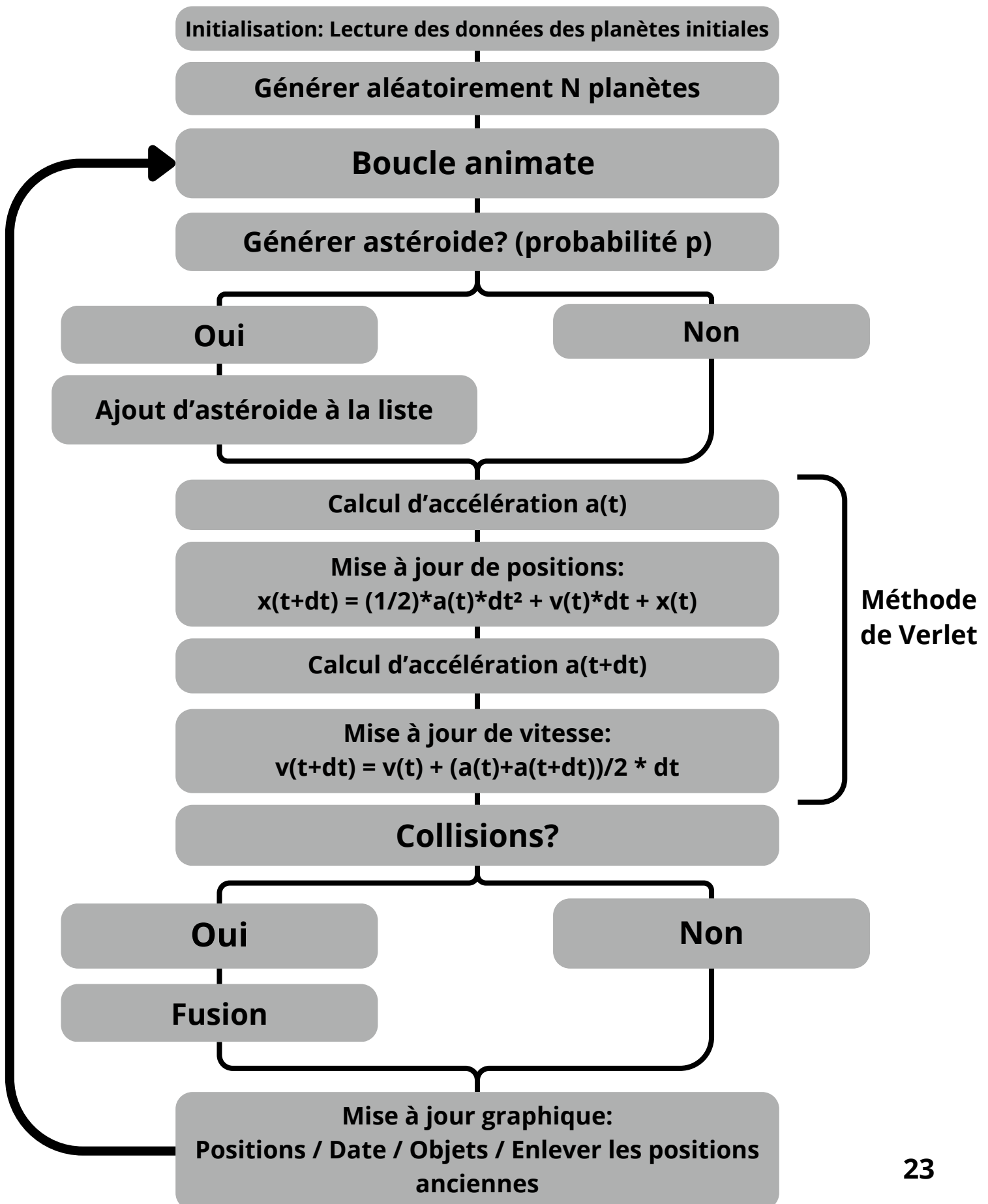
### Sorties :

- Animation temps réel
- Détection collisions
- Affichage date
- Caractéristiques nouveaux objets

### 3.2) Schéma du processus d'analyse des données



### 3.3) Algorithme de la boucle de simulation



## 4) Structure de données

Chaque objet céleste est représenté par un dictionnaire de forme:

```
{  
    "nom": string,           # Nom de l'objet  
    "r": float,              # Rayon (m)  
    "m": float,              # Masse (kg)  
    "p": np.array([[X, Y]]), # Historique des positions  
    "v": np.array([Vx, Vy]), # Vitesse actuelle  
    "color": string          # Couleur pour affichage  
}
```

Cette structure permet un accès direct et efficace aux propriétés de chaque objet. Les dictionnaires sont ensuite rangés dans une liste **planets**.

## 5) Lois utilisés dans la simulation

Après avoir défini l'architecture logicielle et la structure des données, nous allons détailler les lois physiques qui contrôlent les interactions entre les objets célestes au sein de la boucle de simulation.

### 5.1) Loi de la gravitation universelle de Newton

Deux corps  $i, j$  de masse  $m_1$  et  $m_2$  séparés par une distance  $r$  exercent l'un sur l'autre une force attractive dont la formule est :



$$\vec{F} = -G \frac{m_i \cdot m_j}{r^2} \vec{u}$$

Avec  $\vec{u}$  : Vecteur unitaire allant de i vers j

$\vec{F}$  : Force gravitationnelle exercée par le corps i sur j

G : Constante gravitationnelle

$m_j, m_i$  : masses des corps

r : distance entre les deux corps

Le vecteur unitaire s'exprime en fonction du vecteur position :

$$\vec{u} = -\frac{\vec{r}}{r}$$

Avec  $\vec{r} = \vec{r}_i - \vec{r}_j$  : Vecteur position allant de j vers i

$r = \|\vec{r}\|$  : Norme du vecteur position

En substituant dans la loi de gravitation :

$$\vec{F} = G \frac{m_i \cdot m_j}{r^3} \vec{r}$$

## 5.2) Deuxième loi de Newton (Principe fondamental de la dynamique)

Cette loi mentionne que la somme des forces appliquées sur un objet est égale au produit de sa masse et son accélération. Mathématiquement sa formule est :

$$\sum \vec{F} = m_j \vec{a}$$

Cette formule nous aide à calculer l'accélération des objets dans notre simulation en remplaçant la force par sa formule :

$$\vec{a}_j = \frac{1}{m_j} \sum_{i=1}^n G \frac{m_i m_j}{r^3} \vec{r} = \sum_{i=1}^n G \frac{m_i}{r^3} \vec{r} = \sum_{i=1}^n G \frac{m_i}{r^3} (\vec{r}_i - \vec{r}_j)$$

Avec  $\vec{a}_j$  : l'accélération du corps j

n : le nombre des corps exerçant une force sur j

Implémentation dans le code :

```
def acceleration(j):
    a = np.array([0.0, 0.0])
    for i in range(len(planets)):
        if i != j:
            ai = G * planets[i]["m"] / distance(i, j) ** 3 * (planets[i]["p"][-1] - planets[j]["p"][-1])
            a += ai
    return a
```

### 5.3) Méthode d'intégration de Verlet

La méthode de Verlet est un algorithme d'intégration numérique qui résout les équations du mouvement en préservant mieux l'énergie totale du système que la méthode d'Euler simple.

#### **Formules:**

Etape 1: Calcule de la nouvelle position

$$\vec{r}_j(t + \Delta t) = \frac{1}{2} \vec{a}_j(t) \times \Delta t^2 + \vec{v}_j(t) \times \Delta t + \vec{r}_j(t)$$

Etape 2: Calcule de la nouvelle accélération

$$\vec{a}_j(t + \Delta t) = \sum G \frac{m_i}{r^3} [\vec{r}_i(t + \Delta t) - \vec{r}_j(t + \Delta t)]$$

Etape 3: Mise à jour de la vitesse pour l'utiliser dans la position prochaine

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{1}{2} [\vec{a}(t) + \vec{a}(t + \Delta t)] \times \Delta t$$

Implémentation dans le code :

```
a = np.array([acceleration(j) for j in range(len(planets))])

for n in range(len(planets)):
    pos = a[n]/2 * dt**2 + planets[n]["v"]*dt + planets[n]["p"][-1]
    planets[n]["p"] = np.append(planets[n]["p"], [pos], axis=0)

a_new = np.array([acceleration(j) for j in range(len(planets))])

for k in range(len(planets)):
    vitesse = planets[k]["v"] + ((a[k] + a_new[k]) * dt) / 2
    planets[k]["v"] = vitesse
```

## 5.4) Conservation de quantité de mouvement

Dans un système isolé (sans forces extérieures), la quantité de mouvement totale se conserve lors d'une collision. Ce qui nous donne une relation essentielle pour gérer les contacts entre les corps correctement:

$$\vec{p}_{avant} = \vec{p}_{apres}$$

avec  $\vec{p}_{avant}$  et  $\vec{p}_{apres}$  es quantités de mouvements totales du système avant et après collision.

Or, la quantité de mouvement d'un corps est définie par :  $\vec{p} = m\vec{v}$

Ce qui donne pour collision de deux corps i et j:

$$m_j \vec{v}_j + m_i \vec{v}_i = (m_i + m_j) \vec{v}_{new}$$

$$\Rightarrow \vec{v}_{new} = \frac{m_j \vec{v}_j + m_i \vec{v}_i}{(m_i + m_j)}$$

Implémentation dans le code :

```
def fusion(surv, disparait):

    m1 = float(planets[surv]["m"])
    m2 = float(planets[disparait]["m"])

    v1 = np.array(planets[surv]["v"])
    v2 = np.array(planets[disparait]["v"])

    r1 = planets[surv]["r"]
    r2 = planets[disparait]["r"]

    m_new = m1 + m2
    v_new = (m1 * v1 + m2 * v2) / m_new
```

### 5.5) Conservation du volume (approximation)

**Hypothèse:** Lors de la fusion, on suppose que le volume total est conservé.

**Formule:**  $V_t = V_1 + V_2$

Le volume d'une sphère est donné par :  $V = \frac{4}{3}\pi r^3$

En supposant que les planètes sont presque des sphères, cette relation nous permet d'obtenir le nouveau rayon de l'objet issue de la fusion de deux corps.

$$\frac{4}{3}\pi r_f^3 = \frac{4}{3}\pi r_1^3 + \frac{4}{3}\pi r_2^3$$

$$\Rightarrow r_f^3 = r_1^3 + r_2^3$$

$$\Rightarrow r_f = \sqrt[3]{r_1^3 + r_2^3}$$

Implémentation dans le code:

```
planets[surv]["r"] = (r1 ** 3 + r2 ** 3) ** (1 / 3)
```

# Conclusion

Grâce à ce projet, nous avons eu l'opportunité de découvrir le domaine de l'astrophysique et l'espace en utilisant le langage Python.

Nous avons pu étudier des lois liées à ce domaine, telles que la 3ème loi de Kepler, la relation entre la température et la distance des exoplanètes à leur étoile, ainsi que la relation entre la masse et le rayon des étoiles. En utilisant des données réelles issues de sources fiables, nous avons démontré la validité de ces lois en effectuant des régressions linéaires, qui nous ont permis de retrouver les relations théoriques attendues.

En parallèle, nous avons développé une simulation numérique du système solaire, basée sur l'algorithme de Verlet et les données initiales réelles de la NASA, en ajoutant un aspect probabiliste et aléatoire, ce qui nous a permis d'avoir une idée sur le fonctionnement du système solaire.

Enfin, ce travail nous a aidé à approfondir nos connaissances en astrophysique, et également de développer des compétences en traitement et analyse de données, en modélisation numérique, et en programmation Python. Il nous a aussi appris à adopter un regard critique sur nos résultats en identifiant les limites et les approximations de nos modèles, ce qui est une compétence essentielle dans les démarches scientifiques.

# Annexes

## Annexe 1: Sources des données utilisés

- **Loi de Kepler**

Données des lunes de Jupiter :

<https://gist.github.com/adrn/8191304#file-data-csv-L3>

Données des planètes du système solaire :

[https://dataherb.github.io/flora/planets\\_in\\_solar\\_system/](https://dataherb.github.io/flora/planets_in_solar_system/)

- **Relation entre température et distance**

<https://exoplanetarchive.ipac.caltech.edu/cgi-bin/TblView/nph-tblView?app=ExoTbls&config=PS>

Archive officiel de la NASA/Caltech contenant plusieurs propriétés physiques des exoplanètes confirmés

- **Relation entre masse et rayon (étoiles)**

<https://vizier.cds.unistra.fr>

Catalogue astronomique de l'Observatoire de Strasbourg (CDS) regroupant les masses et rayons de plusieurs étoiles.

- **Simulation**

<https://ssd.jpl.nasa.gov/horizons/app.html#/>

Système Horizons du Jet Propulsion Laboratory (NASA) fournissant les positions et vitesses précises des corps du système solaire à un instant.

## **Annexe 2: Sources des nouvelles relations théoriques utilisées**

- **Relation entre température et distance**

[https://fr.wikipedia.org/wiki/Temp%C3%A9rature\\_d%27%C3%A9quilibre\\_%C3%A0\\_la\\_surface\\_d%27une\\_plan%C3%A8te](https://fr.wikipedia.org/wiki/Temp%C3%A9rature_d%27%C3%A9quilibre_%C3%A0_la_surface_d%27une_plan%C3%A8te)

- **Relation entre masse et rayon (étoiles)**

[https://fr.wikipedia.org/wiki/Relation\\_masse-rayon](https://fr.wikipedia.org/wiki/Relation_masse-rayon)

- **Méthode de Verlet**

<https://femto-physique.fr/analyse-numerique/methode-de-verlet.php>